Experiment No. 5
Implement Item-based Nearest neighbor recommendation
Date of Performance:
Date of Submission:
Marks:
Sign:

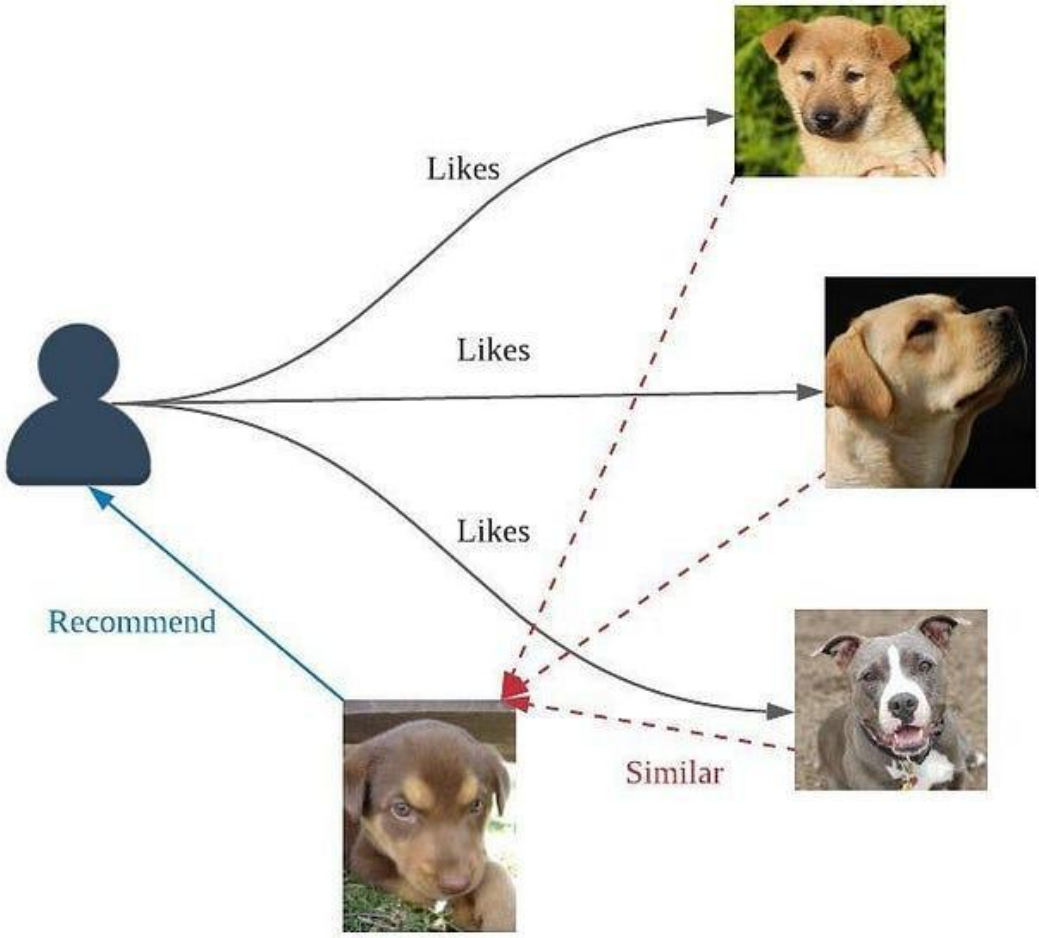
Aim: Implement Item-based Nearest neighbor recommendation.

Objective: Able to interpret and implement Item based Nearest neighbor recommendation.

Theory:

Item-based Collaborative Filtering

Item-based collaborative filtering uses the rating of co-rated item to predict the rating on specific item.



For example, we want to predict rating of user 2 on item 2.

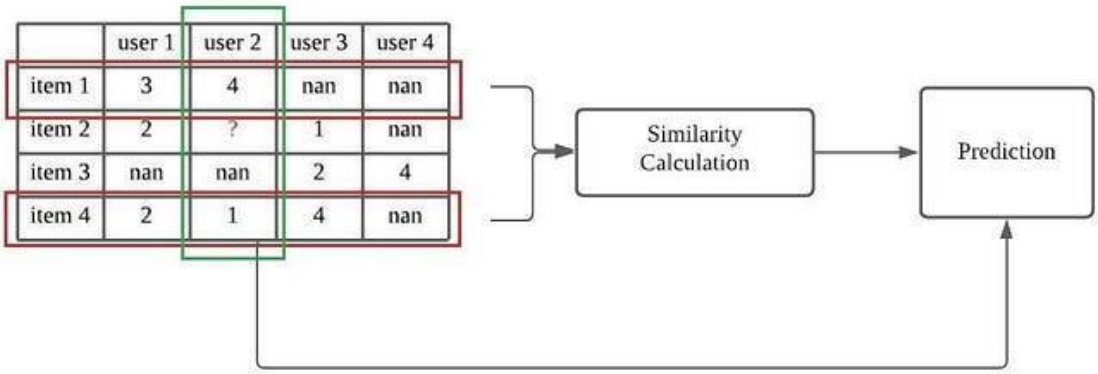
	user 1	user 2	user 3	user 4
item 1	3	4	nan	nan
item 2	2	?	1	nan
item 3	nan	nan	2	4
item 4	2	1	4	nan

To predict the rating,

1. Find the co-rated items of user 2, which is item 1 and item 4
2. Calculate the similarity between item 2 and item 1, 4
3. Calculate the prediction based on the similarity and co-rated rating, such that

$$prediction_{u,i} = \frac{\sum_n \omega_{i,n} * r_{u,n}}{\sum_n |\omega_{i,n}|}$$

where w is the similarity, and r is the rating value.



Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample User-Item Rating Matrix
data = {
    'Item1': [5, 4, 0, 1],
    'Item2': [3, 0, 0, 1],
    'Item3': [4, 0, 0, 1],
    'Item4': [4, 0, 0, 5],
    'Item5': [0, 1, 5, 4]
}

users = ['User1', 'User2', 'User3', 'User4']
df = pd.DataFrame(data, index=users)

print("User-Item Matrix:")
print(df)

# Step 1: Compute Item Similarity (Cosine Similarity)
item_similarity = cosine_similarity(df.T)
item_similarity_df = pd.DataFrame(item_similarity, index=df.columns,
columns=df.columns)

print("\nItem Similarity Matrix:")
print(item_similarity_df)

# Step 2: Predict Missing Ratings using Weighted Average

def predict_rating(user, target_item):
    user_ratings = df.loc[user]
```

```
    similarities = item_similarity_df[target_item]

    numerator = 0
    denominator = 0

    for item in df.columns:
        if user_ratings[item] > 0 and item != target_item:
            numerator += similarities[item] * user_ratings[item]
            denominator += abs(similarities[item])

    if denominator == 0:
        return 0
    return numerator / denominator

# Example: Predict rating for User2 on Item2
predicted_rating = predict_rating('User2', 'Item2')

print("\nPredicted rating of User2 for Item2:", predicted_rating)

# Recommend items for User2
unrated_items = [item for item in df.columns if df.loc['User2', item] == 0]

recommendations = {}
for item in unrated_items:
    recommendations[item] = predict_rating('User2', item)

# Sort recommendations
recommended_items = sorted(recommendations.items(), key=lambda x: x[1],
reverse=True)

print("\nRecommended Items for User2:")
```

```
for item, score in recommended_items:
    print(item, ":", score)
```

Conclusion:

In this experiment, Item-Based Nearest Neighbor recommendation was implemented using collaborative filtering. The similarity between items was calculated using cosine similarity, which helped identify items that are similar based on user ratings. Missing ratings were predicted using a weighted average of ratings from similar items. This approach provides more stable and scalable recommendations compared to user-based methods, especially when the number of users is large. Hence, item-based collaborative filtering is widely used in modern recommendation systems.